LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL IV & V
The Core Of OOP & Encapsulation

Disusun Oleh:

Danu Dimas Putra 105224003

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Dalam permintaan THT, manajemen perhotelan membutuhkan pembaruan arsitektur sistem reservasi besar-besaran karena sering terjadi bug pada operasional, seperti tamu yang melebihi batas muatan kamar hingga penyalahgunaan kode voucher. Untuk mengatasi hal tersebut, pembaruan arsitektur sistem memerlukan prinsip Encapsulation dengan memastikan seluruh data pada sistem, mulai dari tipe kamar hingga status ketersediaan, terlindungi dari akses langsung yang tidak sah dan manipulasi yang merugikan.

Blueprint dari kelas KamarHotel perlu dirancang dengan menerapkan prinsip Encapsulasi secara ketat agar setiap datanya aman dan terlindungi. Lalu juga perlu menerapkan *Method Overloading* agar sistem mampu menangani skenario pemesanan dan perhitungan tagihan secara fleksibel namun tetap terkontrol oleh aturan validasi seperti standarisasi harga minimum dan verifikasi penggunaan diskon.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	nomorKamar	String	Untuk menyimpan huruf dari data nomor kamar dan salah satu variabel yang berada sebagai atribut di dalam class.
2	tipeKamar	String	Untuk menyimpan huruf dari data tipe kamar dan juga salah satu variabel yang berada sebagai atribut di dalam class.
3	kapasitasMaksimal	int	Untuk menyimpan nilai angka bulat dari data kapasitas maksimal dan juga salah satu variabel yang berada sebagai atribut di dalam class.
4	hargaPerMalam	double	Untuk menyimpan nilai desimal dari data harga per malamnya dan salah satu variabel yang berada sebagai atribut di dalam class.
5	isTersedia	boolean	Yang menyimpan nilai true (ketika tersedia kamarnya) dan false (ketika kamarnya terisi) dan salah satu variabel yang berada sebagai atribut di dalam class.
6	nomKamar	String	Sebagai variabel dari parameter *constructor* yang menyimpan nilai huruf dan nilainya akan di-*assign* ke variabel atribut di dalam *constructor.*
7	tipeKamar	String	Sebagai variabel dari parameter *constructor* yang menyimpan nilai huruf dan nilainya juga akan di-*assign* ke variabel atribut di dalam *constructor.*
8	nomorK	String	Sebagai variabel dari parameter *constructor* yang menyimpan nilai huruf dan nilainya akan di-*assign* ke variabel atribut di dalam *constructor.*

9	tipe	String	Sebagai variabel dari parameter *constructor* yang menyimpan nilai huruf dan nilainya akan di-*assign* ke variabel atribut di dalam *constructor.*
10	kapasitas	int	Sebagai variabel dari parameter *constructor* yang menyimpan nilai angka bulat dan nilainya akan di-*assign* ke variabel atribut di dalam *constructor.*
11	hargaPM	double	Sebagai variabel dari parameter *constructor* yang menyimpan nilai desimal dan nilainya akan di-*assign* ke variabel atribut di dalam *constructor.*
12	jumlahTamu	int	Sebagai variabel dari parameter *procedural* yang menyimpan nilai angka bulat dari data jumlah tamu yang akan diberikan saat di main dan nilainya akan di-*assign* ke variabel atribut di dalam *procedural.*
13	kodeVoucher	String	Sebagai variabel dari parameter *procedural* yang menyimpan nilai huruf dari data kode voucher yang akan diberikan saat di main dan nilainya akan di-*assign* ke variabel atribut di dalam *procedural.*
14	kamar	KamarHotel	Sebagai objek yang memliki blueprint dan dapat di akses setiap datanya kecuali atribut langsung (karena encapsulation) di dalam kelas Kamar Hotel .
15	Kamar2	KamarHotel	Sebagai objek yang memliki blueprint dan dapat di akses setiap datanya kecuali atribut langsung (karena encapsulation) di dalam kelas Kamar Hotel.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	KamarHotel()	*constructor*	Untuk inisialiasi setiap nilai atribut yang akan di-assign datanya saat objek dibuat.
2	getNomorKamar()	*procedural*	Untuk pengembalian nilai atribut nomor kamar yang akan memungkinkan kelas lain membaca nilai dari atribut private tanpa bisa mengubah nilainya secara langsung.
3	getTipeKamar()	*procedural*	Juga untuk pengembalian nilai atribut tipe kamar (nilai huruf) yang berguna untuk baca saja tanpa bisa diubah nilainya.
4	getKapasitasMaksimal()	*procedural*	Untuk pengembalian nilai atribut kapasitas maksimal (nilai angka bulat) yang berguna untuk baca saja tanpa bisa diubah nilainya.
5	getHargaPerMalam()	*procedural*	Untuk pengembalian nilai atribut harga per malamnya (nilai desimal) yang berguna untuk baca saja tanpa bisa diubah nilainya.

6	isTersedia()	*procedural*	Untuk pengembalian nilai atribut dari Boolean (true/false) yang berguna untuk baca saja tanpa bisa diubah nilainya.
7	setNomorKamar()	*procedural*	Untuk dapat perubahan nilai dari nomor kamar dan membuat atribut jadi dapat di-set nilainya secara publik.
8	setTipeKamar()	*procedural*	Juga untuk dapat merubah nilai dari tipe kamar dan membuat atribut jadi dapat di-set nilainya secara publik serta untuk memvalidasi nilai yang dapat dirubahnya ("Reguler", "Premium", atau "Suite").
9	setKapasitasMaksimal()	*procedural*	Untuk dapat perubahan nilai dari nomor kamar dan membuat atribut jadi dapat di-set nilainya secara publik.
10	setHargaPerMalam()	*procedural*	Untuk dapat perubahan nilai dari harga per malam dan membuat atribut jadi dapat di-set nilainya secara public serta memvalidasi jika harga di set kurang dari 50000, nilai harganya akan dipaksa menjadi 50000.
11	pesanKamar()	*functional*	*Method* untuk pemesanan kamar dengan validasi atribut isTersedia, jika true mengartikan kamar kosong dan dapat dipesan untuk diisi.
12	batalPesan()	*functional*	*Method* untuk membuat nilai atribut dari isTersedia kembali true karena kamarnya tidak jadi diisi (nilai false)
13	hitungTotalBayar()	*functional*	*Method* untuk menghitung total tagihan dengan jumlah harinya.
14	display()	*functional*	*Method* untuk menampilkan/mencetak setiap nilai dari atribut dengan rapih dan detail.

IV. Dokumentasi dan Pembahasan Code

```java
public class KamarHotel {
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;

    KamarHotel(String nomKamar, String tipeKamar, int kapasitasMaksimal) {
        this.nomorKamar = nomKamar;
        this.tipeKamar = tipeKamar;
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0;
        this.isTersedia = true;
    }

    KamarHotel(String nomorK, String tipe, int kapasitas, double hargaPM) {
        this.nomorKamar = nomorK;
        this.tipeKamar = tipe;
        this.kapasitasMaksimal = kapasitas;
        this.hargaPerMalam = hargaPM;
        this.isTersedia = true;
    }


    public String getNomorKamar() {
        return nomorKamar;
    }

    public String getTipeKamar() {
        return tipeKamar;
    }

    public int getKapasitasMaksimal() {
        return kapasitasMaksimal;
    }

    public double getHargaPerMalam() {
        return hargaPerMalam;
    }

    public boolean isTersedia() {
        return isTersedia;
    }
```

```java
public void setNomorKamar(String nomorKamar) {
    this.nomorKamar = nomorKamar;
}

public void setTipeKamar(String tipeKamar) {
    if (tipeKamar == "Reguler") {
        this.tipeKamar = tipeKamar;

    }
    else if (tipeKamar == "Premium") {
        this.tipeKamar = tipeKamar;

    }
    else if (tipeKamar == "Suite") {
        this.tipeKamar = tipeKamar;
    }
    else {
        this.tipeKamar = "Reguler";
        //System.out.println("Tipe kamar akan dipaksa ke reguler!");
    }
}

public void setKapasitasMaksimal(int kapasitasMaksimal) {
    this.kapasitasMaksimal = kapasitasMaksimal;
}

public void setHargaPerMalam(double hargaPerMalam) {
    if (hargaPerMalam < 50000) {
        System.out.println("Tidak boleh dibawah 50000! Harga akan diset secara paksa: " + 50000);
        this.hargaPerMalam = 50000;
    }
    else {
        this.hargaPerMalam = hargaPerMalam;
    }
}
```

```java
public void pesanKamar() {
    if (isTersedia == true) {
        this.isTersedia = false;
        System.out.println(x: "Kamar berhasil diisi!");
    }
    else {
        System.out.println(x: "Kamar telah terisi!");
    }

}

public void pesanKamar(int jumlahTamu) {
    if (isTersedia == true && jumlahTamu <= kapasitasMaksimal) {
        this.isTersedia = false;
        System.out.println(x: "Kamar berhasil diisi!");
    }
    else {
        this.isTersedia = true;
        System.out.println(x: "Kapasitas tidak mencukupi, Silahkan pesan kamar tambahan");
    }
}

public void batalPesan() {
    this.isTersedia = true;
}

public void hitungTotalBayar(int jumlahMalam) {
    this.hargaPerMalam *= jumlahMalam;
}

public void hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
    if (kodeVoucher == "PROMO" && jumlahMalam >= 3) {
        this.hargaPerMalam *= jumlahMalam;
        double diskon = this.hargaPerMalam * 0.20;
        //double total = this.hargaPerMalam - diskon;
        this.hargaPerMalam -= diskon;
    }
    else {
        this.hargaPerMalam *= jumlahMalam;
        System.out.println(x: "Penggunaan Voucher ditolak");
    }
}

public void display() {
    System.out.println("Nomor Kamar : " + this.nomorKamar);
    System.out.println("Tipe Kamar : " + this.tipeKamar);
    System.out.println("Kapasitas : " + this.kapasitasMaksimal);
    System.out.println("Harga Tagihan : " + this.hargaPerMalam);
    System.out.println("Status Kamar Terisi : " + this.isTersedia + "\n");
}
```

```java
//Danu Dimas Putra 105224003

public class App {
    Run | Debug
    public static void main(String[] args) throws Exception {
        //System.out.println("Hello, World!");
        KamarHotel kamar = new KamarHotel(nomKamar: "105", tipeKamar: "Premium", kapasitasMaksimal: 3);
        KamarHotel kamar2 = new KamarHotel(nomorK: "224", tipe: "Suite", kapasitas: 2, hargaPM: 51000);

        kamar.setTipeKamar(tipeKamar: "Presidential");
        kamar.setHargaPerMalam(-10000);

        kamar2.pesanKamar(jumlahTamu: 4);
        kamar2.pesanKamar(jumlahTamu: 2);

        kamar2.pesanKamar();

        kamar.hitungTotalBayar(jumlahMalam: 2,kodeVoucher: "PROMO");
        kamar2.hitungTotalBayar(jumlahMalam: 4,kodeVoucher: "PROMO");

        kamar.display();
        kamar2.display();



    }
}
```

```
PS D:\Praktikum_PBO\praktikum_pbo-Modul5\praktikum_pbo-Mod5-tugasTHT>  & 'C:\Progr
le-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\Praktikum_PBO\prak
Tidak boleh dibawah 50000! Harga akan diset secara paksa: 50000
Kapasitas tidak mencukupi, Silahkan pesan kamar tambahan
Kamar berhasil diisi!
Kamar telah terisi!
Penggunaan Voucher ditolak
Nomor Kamar : 105
Nomor Kamar : 105
Tipe Kamar : Reguler
Kapasitas : 3
Harga Tagihan : 100000.0
Harga Tagihan : 100000.0
Status Kamar Terisi : true

Nomor Kamar : 224
Tipe Kamar : Suite
Kapasitas : 2
Harga Tagihan : 163200.0
Status Kamar Terisi : false
```

A. Pembahasan Code

Untuk setiap sintaks di class KamarHotel yang akan menjadi sebuah blueprint dari objek yang dibuat, perlu untuk membuat atribut untuk setiap variablenya. Dengan access modifier “private” untuk sebuah objek nantinya yang melakukan enkapsulasi dan menyembunyikan data atau nilainya dari dunia luar. Dari setiap sintaks tersebut perlu dilakukan *treatment* enskapsulasi yang akan membutuh getter sebagai pembaca nilainya dan setter yang dapat merubah nilai dari atributnya secara public serta dapat dengan validasi tertentu.

Setelah pembuatan atribut perlu untuk membuat *constructor* sebagai inilisiasi dari bentuk objeknya yang akan meng-assign setiap nilai ke atribut. Untuk pembuatan *constructor*, dibuat dengan *overloading* yang membuat dua *constructor* dengan nama yang sama namun berbeda parameter. Selanjutnya untuk enskapsulasi perlu untuk membuat getter yang akan mengambil serta mengembalikan setiap nilainya ketika dipanggil dan setter yang dapat merubah nilai dari setiap atribut di-set secara publik (luar class). Enskapsulasi ini gunanya agar semua atribut tidak dapat diakses langsung dari luar kelas dan perlu diakses dengan metode tertentu yaitu dapat dengan getter dan setter itu. Pembuatan setter juga dapat disertai dengan validasi seperti yang diminta dari saat setTipeKamar yang hanya memvalidasi untuk nilai "Reguler", "Premium", atau "Suite" yang seharusnya dalam validasi tersebut perlu untuk equalsIgnoreCase yang tidak memperdulikan besar kecil hurufnya saat men-set nilainya seperti yang diminta. setHargaPerMalam juga perlu validasi ketika harganya di-set kurang dari 50000 maka nilai di atributnya akan di-set secara paksa 50000.

Lalu untuk pembuatan *method* ketika memesan kamar, pesanKamar() akan memvaliadasi berdasarkan sebuah nilai Boolean yang jika true menandakan objek dari sebuah kamar belum terisi dan dapat diisi, lalu jika false kamar tersebut sudah diisi. Untuk yang disertai dengan parameter jumlahTamu, perlu validasi dengan gerbang logika AND agar saat validasi dapat memerikasa “apakah kamar tersedia dan jumlah

tamunya mencukupi kapasitasnya ?" jika terpenuhi maka kamar dapat diisi dan sebaliknya jika tidak terpenuhi. *Method* batalPesan() juga perlu untuk membuat kamar kembali tersedia (true) saat pemesanan dibatalkan. Selanjutnya perlu untuk membuat perhitungan total tagihannya seperti yang diminta yang juga menggunakan *method overloading* mirip seperti *overloading* di *constructor*. Dengan parameter yang berbeda, hitungTotalBayar() dengan parameter jumlahMalam hanya akan mengkalikan hargaPerMalam yang berupa standar harganya dengan jumlah malamnya. Untuk yang disertai dengan parameter kodeVoucher akan divalidasi terlebih dahulu agar dapat diskon pemotogan total tagihannya. Jika jumlah malamnya lebih dari sama dengan 3 dan nilai huruf "PROMO" itu benar maka akan dapat pemotongannya sebesar 20% dan sebaliknya jika tidak terpenuhi.

Selanjutnya untuk menampilkan setiap nilai yang terdapat pada atribut perlu untuk membuat *method* display() agar seluruh informasi atau profil kamar tersebut dapat diketahui. Terakhir, buat objek dua kamar di main lalu inisialisasikan parameter pada objek dan dapat di tes sesuai permintaan serta objek enscapsulasi telah berhasil dibuat.

V. Kesimpulan

Program pada sistem reservasi yang diperbarui ini, menghasilkan sebuah bentuk yang berprinsipkan enskapsulasi dengan setiap nilainya tidak dapat langsung diakses secara langsung. Dalam prinsip enskapsulasi, setiap data atribut pada blueprint objek (class) aksesnya "private" dan hanya bisa digunakan di dalam class tersebut yang membuat setiap data dan nilainya aman dari perubahan secara langsung diluar class tersebut. Perubahan yang dapat dilakukan perlu menggunakan metode tertentu, Dengan cara ini, data dilindungi dari modifikasi langsung yang tidak diinginkan dari luar class, dan pengembang dapat menyisipkan logika validasi atau aturan tertentu ketika data tersebut diubah. Untuk metode tertentu yang digunakan dapat berupa getter dan setter, getter sebagai pembaca dan yang mengembalikan atau mengambil nilai dari suatu atributnya dan setter yang dapat menggubah nilai dari atributnya dan membuatnya dapat diakses secara publik serta dari setter yang dibuat dapat menggunakan validasi agar konsiten pada nilai dari atribut objek kamar yang ditentukan.

Pembuatan *constructor* dalam class juga berfungsi untuk menginisialisasikan setiap nilai di atribut saat objek dibuat. Metode lainnya dalam class berfokus pada control if-else untuk kondisi tertentu seperti pesan kamar yang jika tersedia (true) tandanya kamar siap untuk diisi dan false untuk kamar yang telah terisi. Metode ini seperti aksi atau perilaku dari objek. Lalu pembuatan objek kamar di main perlu inisialisasi setiap nilai di parameternya agar pembuatan objeknya dapat memiliki informasi yang jelas sesuai blueprint di *constructor*-nya. Dan pada main atribut tidak dapat diakses secara langsung karena prinsip enskapsulasi, panggilkan getter untuk membacanya dan setter untuk akses secara publiknya.

VI. Daftar Pustaka

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). *Modul Praktikum Pemograman Berorientasi Objek; The Core Of OOP*. Universitas Pertamina.

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). *Modul Praktikum Pemograman Berorientasi Objek; Encapsulation*. Universitas Pertamina.